Software Requirements Specification

for

Musiculous Ai.

Version 1.0 Waiting for Approval

Prepared by Khittitaj Bunupuradah

Kasetsart University

29/1/2026

Table of Contents

- Table of Contents
- 1. Introduction 1
  - 1.1 Purpose 1
  - 1.2 Product Scope 1
  - 1.3 Document Convention 1
  - 1.4 Intended Audience 2
- 2. Overall Description 2
  - 2.1 Product Perspective 2
  - 2.2 Product Functions 2
  - 2.3 User Classes and Characteristics 2
  - 2.4 Stakeholder Role and Concern Analysis 4
  - 2.5 Operating Environment 5
  - 2.6 Design and Implementation Constraints 5
  - 2.7 Assumptions and Dependencies 5
  - 2.8 Open Issues 6
- 3. External Interface Requirements 6
  - 3.1 User Interfaces 6
  - 3.2 Software Interfaces 7
  - 3.3 Communications Interfaces 7
- 4. System Features 7
  - 4.1 Feature: User Stories 7
  - 4.2 Feature: User Authentication and Profile Management ... 8
  - 4.3 Feature: AI-Powered Music Generation 9
  - 4.4 Feature: Music Library & Social Distribution 9
- 5. Other Nonfunctional Requirements 12
  - 5.1 Performance Requirements 12
  - 5.2 Safety Requirements 12
  - 5.4 Business Rules 13
- 6. Use Case Modelling 13
  - 6.1 Use Case Diagram 13
  - 6.2 Use Case Descriptions 14
- 7. Other Requirements 18
- Appendix A: Glossary 18
- Appendix B: Analysis Models 19
- Appendix C: References 19
- Revision History 19

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification is to define the functional and non-functional requirements for **Musiculous Ai**, a web-based AI music generation platform. This document covers the system's constraints, user characteristics, and specific feature requirements required to build the MVP, or minimum viable product.

It serves as the authoritative source for:

- **The Developer:** To guide the implementation of the Django backend and AI integration.
- **The Project Assessors:** To verify that the final delivered software meets the initial specifications.

1.2 Product Scope

For Musiculous Ai, these will be the scope for the project. The following will be a brief explanation of what we will implement in this project;

- **User Authentication:** Secure login via Email/Password and Google OAuth 2.0.
- **AI Integration:** Interfacing with external text-to-audio APIs to generate audio clips from text prompts.
- **Library Management:** A persistent database allowing users to save, rename, view, and delete their generated tracks.
- **Distribution:** Functionality for users to download MP3 files and share public links with guest users.
- **responsive Web Interface:** A client-side application accessible via standard desktop web browsers.

1.3 Document Convention

This document follows the formatting conventions listed below:

- **Requirement Identifiers:** Individual functional requirements are identified with a unique ID (example: **FR-1**), and non-functional requirements are identified by their specific category (example: **Performance-1**).
- **Prioritization:** All requirements are assumed to be of high priority unless otherwise stated.
- **Text Styling:** Bold text is used for emphasis or to highlight specific system components and user interface elements.
- **Standard Definitions:** The keyword "shall" indicates a mandatory system requirement, while "should" indicates a desirable but non-mandatory feature.

1.4 Intended Audience

This document is intended for the following audiences:

- **Clients & Project Supporter:** To revise on project details and suggest fixes upfront before start of development.
- **Developers:** To understand the specific implementation details, including how the API works and database schema.
- **Testers:** To design test cases based on the Functional Requirements and Non-Functional Requirements.

1.5 Out-of-Scope

This section covers things that will not be/unlikely to be implemented within the project.

- **Mobile Support:** The website will completely support computer accesses, fitting every screen type on a computer, instead the website will be partly–or not be– supported on mobile devices.
- **Music Editing Software:** The website's main purpose is to use AI to create music only, so the website will not provide music editing software to edit the musical contents and instruments of generated contents.

2. Overall Description

2.1 Product Perspective

Musiculous Ai. is a new, self-contained product that provides registered users with AI-generated functions. Users may pick multiple options for registration; Google Authentication, or basically inputting their standard credentials. The backend's structure is written in Django for general/global acceptance. The backend also contains an AI model that is used to generate music to the users' commands.

2.2 Product Functions

User Account Management - Allows users to register and/or log in via standard credentials or Google OAuth
.
AI Music Generation - Users may input text descriptions (prompts), genre, or reference an artist to generate original musical tracks.

Personal Music Library - Provides a private library where users can view, name, set covers for, and delete their generated songs.

Playback & Control - Website includes a music player with standard controls (Play, Pause, Skip) to preview generated songs.

Sharing & Distribution - Users are in charge of their songs' privacy (Public/Private), link sharing for external listeners, and the ability to download tracks as MP3 files.

2.3 User Classes and Characteristics

Authenticated User

- **Description**: These are the primary users who interact with the AI to create music. They may create an account, or log in to an account using their personal email credentials, or Google Authentication.
- **Characteristics**: They range from hobbyists to content creators who may not have formal musical knowledge/not know how to play music but want to generate original content.
- **Frequency of Use**: Iterative.
- **Privileges**: They have the ability to generate music, manage a private library (edit/delete/rename), and toggle song privacy settings.
- **Technical Expertise**: Expected to have basic web navigation skills and an understanding of how to write AI prompts.

Unauthenticated User ("Guests")

- **Description**: External individuals who receive a shared link to listen to or download a specific song.
- **Characteristics**: Likely casual users who do not wish to create an account but want to consume content shared by a Creator.
- **Frequency of Use**: Occasional/, or One-time.
- **Privileges**: Limited to "Listen-only" access via a public player page. They can play and download shared MP3 files but cannot access any user libraries.
- **Technical Expertise**: Basic web navigation skills are more than enough.

Administrator

- **Description**: Internal staff responsible for overseeing the platform.
- **Characteristics**: Users with technical knowledge who are focused on system health and user management.
- **Frequency of Use**: Daily (for monitoring and maintenance).
- **Privileges**: High-level access to manage user accounts and monitor system-wide resource usage (like AI generation logs).

2.4 Stakeholder Role and Concern Analysis

Stakeholder	Role in Project	Primary Concerns & Goals
Creator (User)	The primary end-user who provides inputs to the AI.	**Goals:** Easy-to-use, high-quality music output, and personal data privacy. **Concerns:** "Will my songs be saved safely?" "Can i use this interface easily?"
Recipient (Guest)	An external party who listens to or downloads shared content.	**Goals:** Instant access to music without friction (no login). **Concerns:** "Do I need an account to listen?" "Is the download link safe?"
Developer	The person building and maintaining the code.	**Goals:** Code maintainability and easy integration with AI APIs. **Concerns:** "Is the AI API reliable?" "Can I update the system without breaking existing features?"
Administrator	Monitors system health and manages users.	**Goals:** System stability and prevention of platform abuse. **Concerns:** "Are users generating inappropriate content?" "Is the server cost scaling too fast?"

2.5 Operating Environment

2.5.1 Client-Side (User's Device)

To access the application, the end-user requires a device capable of running a modern web browser.

- **Hardware:** A standard desktop, or laptop with audio output capabilities (speakers or headphones).
- **Software:** Any modern web browser supporting HTML5 and JavaScript
- **Network:** A stable internet connection with a minimum speed of 5 Mbps to stream audio without buffering.

2.2.2 Server-Side (Hosting Environment)

The backend application requires a Linux-based server environment to host the Django application and manage database connections.

- **Operating System:** Ubuntu or compatible Linux distribution.
- **Runtime Environment:** Python 3.10 or higher with the pip package manager.
- **Database Engine:** PostgreSQL or SQLite.

2.6 Design and Implementation Constraints

Web Framework: The application backend **must** be implemented using the **Django** web framework. This constraint is imposed to ensure compatibility with the course curriculum and grading.

Programming Language: All server-side logic **shall** be written in **Python 3.10** or higher to utilize modern features required by the AI generation tasks.

Architectural Pattern: The system architecture **must** strictly adhere to **Domain-Driven Design** (DDD) principles. The codebase should demonstrate a clear separation between the Domain, Infrastructure, and Application layers.

Database Technology: The system is constrained to use **PostgreSQL** or **SQLite** as the relational database management system.

External API Dependency: The core functionality depends on the availability and rate limits of the external **AI Music Generation API** (that will probably be provided later down in development). The system must handle potential API timeouts or failures.

2.7 Assumptions and Dependencies

Assumptions: Users have basic computer literacy and an internet connection.

- The AI model API used for music generation will remain operational and accessible during system use.

Dependencies: The login system depends on the availability of Google's authentication services.

- **External AI Service:** The core functionality (music generation) is also dependent on an outside AI model

2.8 Open Issues

Issues 1. Library Capacity: Right now the API to generate the song is to be determined, so we cannot clearly speculate the size of the generated songs, and the details.

- **Possible Solve:** Wait until the instructors announce what API, or AI model we will be using.

Issues 2: Copyright Problem: A pretty common problem amongst AI-generated content is copyrights, the creator of a song on Musiculous Ai. may face problems against copyright laws.

- **Possible Solve:** The system should tag all of the generated music to be "For Personal Uses Only" in order to not face copyright consequences.

3. External Interface Requirements

3.1 User Interfaces

3.1.1 Dashboard (Library): The system shall provide a web-based dashboard featuring a navigation sidebar, a central "Music Generation" option on the right bottom, and a "Library" grid view.

3.1.2 Audio Player: A footer-based audio player shall be present, containing Play/Pause buttons, a seek bar, and volume controls.

3.1.3 Design Consistency: The interface shall use a consistent dark-themed aesthetic (to reduce eye pain during creation/listening) and responsive design for mobile and desktop browsers.

3.2 Software Interfaces

3.3.1 Operating System: The application shall be accessible on any OS capable of running a modern web browser (Windows, macOS, Linux).

3.3.2 Database: The system shall interface with a **PostgreSQL** or **SQLite** database via the Django to store user profiles and song metadata.

3.3.3 Google OAuth: The system shall use the Google OAuth 2.0 client library to manage external authentication and retrieve basic user profile info (name/email).

3.3.4 AI Music Generation API: The system shall communicate with an external AI model, sending text prompts and receiving binary audio data.

3.3 Communications Interfaces

3.4.1 Protocol: All communications between the client and server shall be encrypted using **HTTPS** to protect user data.

3.4.2 API Communication: The system shall use the **JSON** format for sending prompt data to the AI engine and receiving metadata back.

4. System Features

4.1 Feature: User Stories

US1. As a User, I want to describe and generate music with AI so that I will have music based on my taste without any effort.

US2. As a User, I want to set a song cover and name my generated songs so that I can know which song is which.

US3. As a User, I want to be able to login into the site so that I can keep my songs on the website.

US4. As a User, I want to be able to login into the site with my google account so that it's more convenient for me to use.

US5. As a User, I want to see my songs, generated or generating, to show up in my library so that I can keep track of my songs.

US6. As a User, I want to add, delete, or look at descriptions of songs in my library so that I can work with my songs after the draft.

US7. As a User, I want to set the status of my songs to either private or public so that only some of my songs are accessible to everyone.

US8. As a User, I want to have the option to download my songs into MP3 files so that I can send them to other people or use it outside of the website.

US9. As a User, I want to have the option to share links of my songs so that other people outside of the website, users or not, can listen to the songs online.

US10. As a User, I want to have basic music controls, similar to the options in Spotify or Youtube Music, so that I can play, pause, or skip my songs.

US11. As a User, I want to sign up/create an account for the website so that I can access features only people signed in could use.

US12. As a User, I want to have the ability to edit my generated songs' name, information, etc. so that I can adjust the information over time.

US13. As an Administrator, I want the ability to remove a song entirely so that I can remove content that violates the terms of service or copyright rules.

US14. As an Administrator, I want to see a dashboard showing the total number of songs generated per user so that I can keep track of users and check for potential botting/spams.

US15. As an Administrator, I want the ability to disable new song generation while keeping the library accessible so that I can prevent errors of failed music creation while the developers maintain the web.

4.2 Feature: User Authentication and Profile Management

Description: This feature allows users to create an identity within the system, ensuring their generated music is saved to their specific account.

- **Functional Requirements:**
  - **FR-01:** The system shall provide an interface for users to authenticate using Google OAuth.
    - **(Ref: US3, UC1)**
  - **FR-02:** The system shall provide a standard login form for email and password authentication.
    - **(Ref: US4, UC1)**
  - **FR-03:** The system shall maintain a secure session for logged-in users until they explicitly log out.
    - **(Ref: US3, UC1)**

4.3 Feature: AI-Powered Music Generation

Description: The core engine that accepts text prompts and converts them into audio files.

- **Functional Requirements:**
  - **FR-04:** The system shall provide a text input field (max 500 characters) for music descriptions.
    - **(Ref: US1, UC1)**
  - **FR-05:** The system shall send the validated prompt to the AI Music API and retrieve the generated audio file.
    - **(Ref: US1, UC1)**
  - **FR-06:** The system shall display an audio player with Play, Pause, and Volume controls immediately after generation.
    - **(Ref: US2, UC1)**

4.4 Feature: Music Library & Social Distribution

Description: Allows users to view, organize, and curate their collection.

- **Functional Requirements:**
  - **FR-07:** The system shall display a grid view of all songs belonging to the authenticated user.
    - **(Ref: US5, UC2)**
  - **FR-08:** The system shall allow users to edit the "Title" and "Cover Image" fields of any song in their library.
    - **(Ref: US6, UC2)**
  - **FR-09:** The system shall provide a "Delete" function that permanently removes the song and its associated file.
    - **(Ref: US7, UC2)**

4.4 Feature: Sharing and Distribution

Description: Mechanisms for exporting music and sharing it with external users.

- **Functional Requirements:**
  - **FR-10:** The system shall provide a toggle switch to set a song's visibility to "Public" or "Private."
    - **(Ref: US8, UC2)**
  - **FR-11:** The system shall generate a unique, shareable URL for any song marked as "Public."
    - **(Ref: US10, UC2)**
  - **FR-12:** The system shall generate a downloadable MP3 file when the user clicks the "Download" button.
    - **(Ref: US9, UC2)**

4.6 Feature: Security & Performance Implementations

Description: Tools for the users' protection, protect users' sensitive information.

- **NFR-5. Data Encryption:** All sensitive user data, like OAuth tokens and email addresses stored in the database shall be encrypted using the **AES-256** standard.
- **NFR-6. Data Encryption:** All data transmission between the client and the server must be secured using **TLS 1.2 or higher**.
- **NFR-7. Password Policy:** If local accounts are used, the system shall enforce a password complexity policy requiring a minimum of 8 characters, including at least one number and one special character.
- **NFR-8. Session Management:** User sessions shall automatically expire and require re-authentication after **30 minutes of inactivity** to prevent unauthorized access on shared devices.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

NFR-1. Response Time: The system shall load the user's music library page in under 5 seconds under normal 4G+ network conditions.

NFR-2. Generation Latency: The AI music generation process shall complete and serve the audio file to the user within 10 minutes of prompt submission.

NFR-3. Concurrency: The system shall support at least 500 concurrent users performing music generation tasks simultaneously without service degradation.

NFR-4. Timeout Handling: If the external AI API does not respond within 90 seconds, the system shall automatically terminate the request and display an error message to the user, freeing up server resources.

NFR-5. Failure Rate: The system shall maintain a successful generation rate of 95% or higher; fewer than 5% of requests shall fail due to internal server timeouts or other problems.

5.2 Safety Requirements

NFR-4. Data Integrity: The system shall ensure that if a browser session is interrupted, no partial or corrupted audio files are saved to the permanent database.

5.3 Software Quality Attributes

NFR-8. Usability: A registered user shall be able to navigate from the homepage to the "Generate" button in no more than 5 distinct mouse clicks.

NFR-9. Reliability: The system shall require a user confirmation dialog before permanently deleting any asset from the library.

NFR-10. Supportability: The system shall be implemented using a modular Django app structure, ensuring the AI integration logic is not intertwined from the user interface logic.

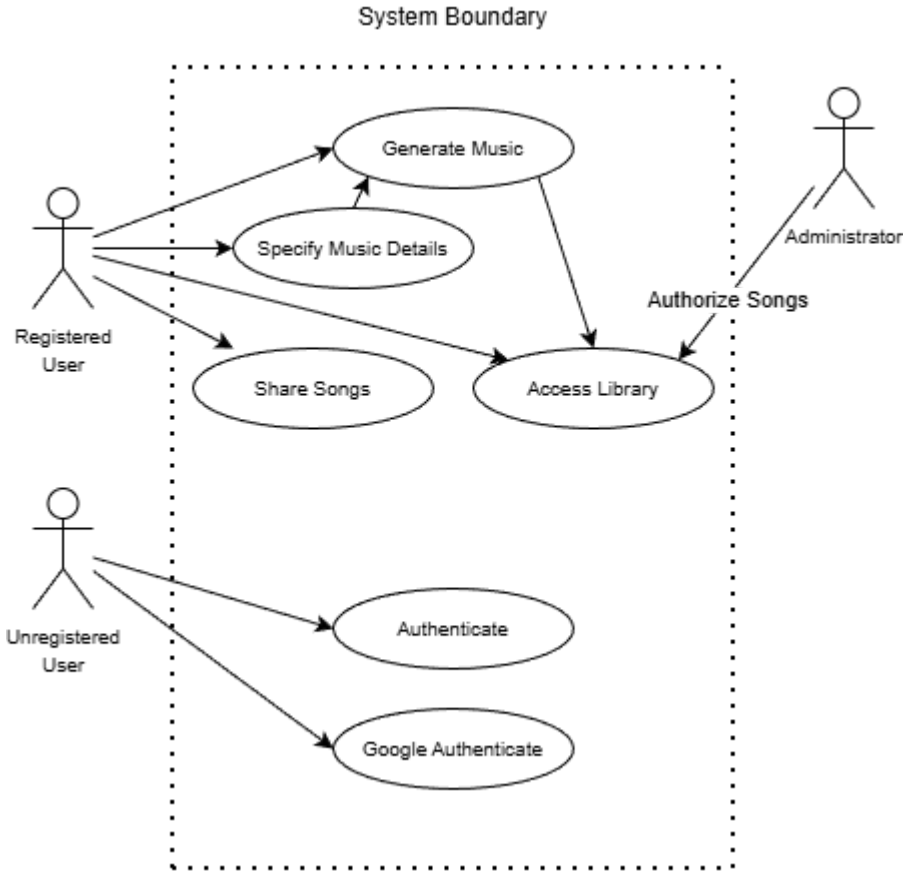
5.4 Business Rules

NFR-011 (Membership Tier): Only registered users shall be permitted to save generated songs to the server; guest users are restricted to "Read-Only" access to shared links.

NFR-012 (IP Ownership): The system shall assign metadata ownership of the generated music file to the user who input the prompt.

6. Use Case Modelling

6.1 Use Case Diagram



6.2 Use Case Descriptions: Fully Dressed Use Case

Use Case: UC1 - Generate Music

- **Primary Actor:** Registered User

- **Goal:** To generate a custom audio track using AI based on a text prompt.

- **Preconditions:**
  - The user is authenticated and logged in.
  - The AI Music Engine API is online and responsive.

- **Main Success Scenario:**
  - The user navigates to the "Create" dashboard.
  - The user enters a descriptive text prompt.
  - The user clicks the "Generate" button.
  - The system validates the prompt length and content.
  - The system sends the request to the AI and displays a progress status.
  - The system receives the audio file, saves it to the database, and updates the User's library.
  - The system notifies the user of a successful generation.

- **Alternative/Exception Flows:**
  - **E1: AI Generation Timeout:** If the AI engine takes longer than 10 minutes, the system stops the request and notifies the user to retry.
  - **E2: Invalid Prompt:** If the user enters an empty prompt or prohibited keywords, the system highlights the error and prevents submission.
  - **E3: Storage Full:** If the server cannot save the file, the system displays an error and prompts the user to delete old songs.

- **Postconditions:**
  - **Success:** The song is stored and playable in the user's private collection. Generation credits or server resources are updated/released.
  - **Failure:** The library remains unchanged; an error message is displayed.

Use Case: UC2 - Manage Library (Editing Songs Detail)

- **Primary Actor:** Creator (Authenticated User)

- **Goal:** To edit songs in the user's library.

- **Preconditions:**
  - The User is logged into the system.
  - The User has generated at least one song.

- **Main Success Scenario:**
  - The User navigates to the "My Library" page.
  - The System displays a grid of songs.
  - The User clicks the "Edit" (Pencil) icon on a specific song card.
  - The System opens a modal displaying the current Title and a "Save" button.
  - The User types a new Title – something like "Cyberpunk Vibes" – and clicks "Save."
  - The System validates the input (checking for forbidden characters or length).
  - The System saves the changes and updates the card display.

- **Alternative/Exception Flows:**
  - **E1: Invalid Input:**
    - The User enters an empty title or one that is too long.
    - The System displays an error message: "Title cannot be empty."
    - The flow returns to the previous step.

- **Postconditions:**
  - **Success:** The song's title or cover image is updated in the database.
  - **Failure:** The library remains unchanged; an error message is displayed.

Use Case: UC3 - Manage Library (Deleting Songs)

- **Primary Actor:** Creator (Authenticated User)

- **Goal:** To delete songs in the user's library.

- **Preconditions:**
  - The User is logged into the system.
  - The User has generated at least one song.

- **Main Success Scenario:**
  - The User navigates to the "My Library" page.
  - The User clicks the "Delete" (Trash Can) icon on a song.
  - The System displays a Confirmation Dialog: "Are you sure? This cannot be undone."
  - The User clicks "Confirm."
  - The System sends a request to the server to delete the record ID.
  - The System removes the song card from the UI and displays a "Song Deleted" toast notification.

- **Alternative/Exception Flows:**
    - **E1: User Cancelled the process:**
        - The User clicks "Cancel" on the confirmation dialog.
        - The System closes the dialog; no data is changed.

- **Postconditions:**
    - **Success:** The song is permanently removed from the database and storage.
    - **Failure:** The library remains unchanged; an error message is displayed.

<u>Use Case: UC4 - Sharing Songs</u>

- **Primary Actor:** Creator (Authenticated User)

- **Goal:** To share songs to other users, authenticated and unauthenticated.

- **Preconditions:**
    - The song currently exists in the User's library with "Private" status.

- **Main Success Scenario (Basic Flow):**
    - The User views their library and identifies a "Private" song.
    - The User clicks the "Share" icon.
    - The System updates the song's visibility status to public in the database.
    - The System generates a unique URL and displays it in a popup.
    - The User clicks "Copy Link."
    - The System copies the URL to the clipboard and shows a "Link Copied" confirmation.

- **Alternative/Exception Flows:**
    - **E1: Reverting to Private:**
        - The song is already "Public."
        - The User clicks the toggle to make it "Private."
        - The System updates the status to PRIVATE.
        - The System invalidates the previous shared link (accessing it will now show a 404 or "Access Denied" error).

- **Postconditions:**
    - **Success:** The song status is updated to "Public," and a unique URL is generated.
    - **Failure:** The library remains unchanged; an error message is displayed.

7. Other Requirements

Appendix A: Glossary

- **API (Application Programming Interface):** A set of rules and protocols that allows different software applications (like the web app and the AI engine) to communicate with each other.
- **DDD (Domain-Driven Design):** A software design approach that focuses on modeling software to match a real-world domain, emphasizing the separation of the "Domain" logic from the "Infrastructure" code.
- **Django:** A high-level Python web framework used for the backend of this project, known for its security and rapid development features.
- **MP3:** A standard digital audio coding format used for storing the generated music files.
- **FR (Functional Requirements):** A requirement that defines how the system works code-wise.
- **NFR (Non-Functional Requirement):** A requirement that defines how the system performs rather than what it does.
- **OAuth 2.0:** The industry-standard protocol for authorization, used in this project to allow users to log in with their Google accounts without sharing their password.
- **Prompt:** The natural language text description provided by the user to guide the AI generation.
- **Seed:** A numerical value used to initialize the AI's random number generator. Using the same seed with the same prompt allows users to reproduce the exact same song.

Appendix B: Analysis Models

User Entity:

- **ID** (Primary Key, UUID)
- **Email** (String, Unique)
- **OAuth_Token** (Encrypted String)
- **Date_Joined** (DateTime)

Song Entity:

- **ID** (Primary Key, UUID)
- **Owner_ID** (Foreign Key -> User)
- **Title** (String, max 100 chars)
- **Description** (Text, max 500 chars)
- **Audio_File_Path** (String, URL to S3/Storage)
- **Public** (Boolean)
- **Created_At** (DateTime)

Appendix C: References

This document refers to the following external standards, documentation, and resources:

1. **IEEE Std 830-1998**: IEEE Recommended Practice for Software Requirements Specifications.
2. **Course Materials**: Principles of Software Engineering, Lecture Slides, Kasetsart University, 2026.
3. **Django Documentation**: Django Software Foundation. Available at: https://docs.djangoproject.com/

Revision History

Name	Date	Reason For Changes	Version